

PRIMERO COPIE EL PROMPT DEL DOC SOLO LE PEDI QUE LO AJUSTE A MIS CLASES

Evaluar Expresión Posfija con Pila



Contexto:

Ya tengo un proyecto desarrollado en clase co...

◆ Ver razonamiento ▾

Para complementar tu proyecto actual con el mínimo de cambios y respetando la estructura de Programación Orientada a Objetos (POO), aquí tienes la propuesta de integración y la explicación de cómo funciona.

Primero me da una breve explicacion

1. ¿Cómo se usa la pila (Stack) durante la evaluación?

El algoritmo de evaluación de expresiones posfijas funciona recorriendo la cadena de izquierda a derecha bajo las siguientes reglas:

1. **Si encuentras un operando (un dígito):** Lo conviertes a un valor numérico (`int` o `float`) y lo guardas en la pila usando `push()` .
2. **Si encuentras un operador (como `+`, `-`, `*`, `/`):** * Sacas el último número de la pila con `pop()` (este será el **operador 2** o **operando derecho**).
 - Sacas el siguiente número con `pop()` (este será el **operador 1** o **operando izquierdo**).
 - Realizas la operación correspondiente según el símbolo.
 - El resultado obtenido lo vuelves a guardar en la pila con `push()` .
3. **Al finalizar el recorrido:** El resultado final de la expresión será el único elemento que quede en la pila. Lo extraes con `pop()` para mostrarlo en tu interfaz gráfica.

Despues me da un posible codigo a integrar en mi proyecto

2. Código a integrar (Sin usar `eval()`)

Puedes encapsular esta lógica dentro de una clase de lógica de tu negocio (por ejemplo, en la clase donde ya tienes el método para convertir de infija a posfija) o crear un método estático/miembro en la misma clase encargada del procesamiento.

```
def evaluar_posfija(expresion_posfija: str) -> float:

    # Creamos una instancia de tu clase Stack existente
    pila = Stack()

    for caracter in expresion_posfija:

        # Si es un dígito, se convierte a entero y se apila
        if caracter.isdigit():
            pila.push(int(caracter))

        # Si es un operador, se desapilan dos elementos y se opera
        elif caracter in ['+', '-', '*', '/']:

            # El primero en salir es el segundo operando (derecho)
            op2 = pila.pop()

            # El segundo en salir es el primer operando (izquierdo)
            op1 = pila.pop()

            # Aplicamos la operación de forma manual sin usar eval()
            if caracter == '+':
                resultado = op1 + op2
            elif caracter == '-':
                resultado = op1 - op2
            elif caracter == '*':
                resultado = op1 * op2
            elif caracter == '/':
```

Opcional: Validar división por cero si fuera necesario

resultado = op1 / op2

El resultado de la operación se guarda en la pila

pila.push(resultado)

Al final, el único elemento en la pila es el resultado definitivo

return pila.pop()

El cual después de analizarlo me pareció razonable después me percate que le faltaba un requerimiento el cual era la potencia que posteriormente agregue para que quedara bien

Después me sugirió como conectarlo a la interfaz gráfica de qt designer

2. ¿Cómo conectar el método con el botón de la Interfaz Gráfica?

En tu archivo donde controlas la ventana de la aplicación (que según tu plantilla se llama `LoadAplicaciones`), debes enlazar el evento del botón asignado para la evaluación en el método `__init__`, e implementar la acción. 

Suponiendo que tu botón de evaluar se llama `btn_evaluar`, la etiqueta para mostrar la posfija es `lbl_posfija`, y la etiqueta de resultado es `lbl_resultado`:

```

# Instanciamos el evaluador
self.evaluador = EvaluadorPosfija()

# Conectamos el botón con el método correspondiente
self.btn_evaluar.clicked.connect(self.procesar_evaluacion)

def procesar_evaluacion(self):
    # 1. Obtenemos la expresión posfija que ya se generó
    expresion_posfija = self.lbl_posfija.text()

    if expresion_posfija:
        # 2. Evaluamos usando nuestra clase orientada a objetos
        resultado_final = self.evaluador.evaluar(expresion_posfija)

        # 3. Mostramos el resultado numérico en el Label
        self.lbl_resultado.setText(str(resultado_final))

```

Del cual tome algunas líneas y posteriormente modifique para enlazar correctamente mis botones.

Al final solo realice pruebas y verifique resultados.